Chapter 31 worked solutions

What Changed, and What Remains?

Exercises 1–4 count causal connections, distinguish architectural changes, and compare what different histories preserve. Exercises 9–12 turn these ideas into evaluation designs.

1. Count permitted pairs

Number the positions in the prefix from 1 to 10. At query position 3, the available causal keys are at positions 1, 2, and 3. A query can read its own position and positions earlier in the prefix. The pair is ordered because the query and key have distinct roles. For example, $(3, 1)$ is permitted, while the reversed pair $(1, 3)$ would look ahead.

With full causal attention, query t reads all t keys from 1 through t. Summing these counts over the ten queries gives

$$1 + 2 + \cdots + 10 = \frac{10 \cdot 11}{2} = 55.$$

With a window of four, query t reads keys from $\max(1, t - 3)$ through t. The current position uses one of the four places. Query 5 is the first to lose a key. Its window starts at 2, so it reads keys 2, 3, 4, and 5. Each of queries 4 through 10 reads exactly four keys.

Query position	Full causal keys	Keys with a window of four
1	1	1
2	1, 2	1, 2
3	1, 2, 3	1, 2, 3
4	1, 2, 3, 4	1, 2, 3, 4
5	1, 2, 3, 4, 5	2, 3, 4, 5
6–10	1 through t	$t - 3$ through t

The earliest three queries have only one, two, and three keys available, respectively. Their key sets are identical under both rules because each fits in the window. With three shorter rows and seven rows of length four, the window admits

$$1 + 2 + 3 + 7 \cdot 4 = 34$$

ordered pairs, including ten self-pairs.

2. Locate an architectural change

Suppose a model reads *the red car*. We can ask it to predict the next word, decide which earlier words it can consult, decide how to store their features, and choose how to carry out the attention calculation. The chapter's history table gives examples that act on each of these decisions.

Design decision	Example from the history table	What the change accomplishes
Training task	BERT's masked recovery, in the 2018 row	The training signal comes from reconstructing selected input tokens. For example, the model can use both *red* and *stopped* to recover a selected token in *the red [MASK] stopped*. This objective teaches the model to use surrounding context.
Stored feature width	Multi-query attention (MQA), in the 2019 row	Multiple query heads share a single key head and a single value head. A retained position needs only one set of key and value features for the layer, which makes each cache entry narrower.
Positions a query can read	Longformer's local and global attention, in the 2020 row	Ordinary positions read nearby positions and designated global positions. Global positions have broader reach. The attention pattern therefore determines which earlier or surrounding features a given query can consult directly.
Attention execution	FlashAttention, in the 2022 row	The implementation processes query and key blocks in tiles and accumulates the statistics needed for softmax. It reduces transfers to and from device memory while evaluating the same attention function up to floating-point rounding.

3. Find two histories with the same summary

Consider two delivery streams. In each stream, the first event records a code. Each later event replaces the current route with a new label.

Event position	History 1	History 2
1	Record code 402	Record code 915
2	Set route A	Set route D
3	Set route B	Set route E
4	Set route C	Set route C

If we keep only the current route, History 1 overwrites A with B and then C. History 2 overwrites D with E and then C. The stored route is C in both cases. This answers the route question. A reader that receives only C has exactly the same input for both histories, even though the correct codes differ.

A summary sufficient for both questions must preserve two pieces of information. Save the **first code** when it arrives and keep that value through subsequent route updates. Save the **current route** and replace it on each assignment. The final summaries are $(402, C)$ and $(915, C)$. A code-only summary would also be insufficient. For example, changing History 1's last route to D would keep code 402 and change the route answer. Both intermediate route sequences can be discarded for these questions. The requirement concerns which distinctions the summary preserves. It does not depend on whether the implementation packs the values into a single record or stores two separate fields.

4. Budget persistent history

At prefix length n, one full-attention layer keeps n entries for use on later steps. Each entry costs four scalars, so that layer keeps $4n$ scalars. There are two such layers, for $8n$ in total. Each recurrent layer carries forward one state of four scalars. Three recurrent layers therefore contribute 12 scalars, regardless of how many updates have been applied.

The persistent history count in the exercise's units is

$$H(n) = 2(4n) + 3(4) = 8n + 12.$$

The state takes 12 scalars from the 92-scalar budget, leaving 80 for the attention entries. One more prefix position uses four additional scalars in each of the two attention layers. At eight scalars per position, $80/8 = 10$ positions fit. Equivalently,

$$8n + 12 \le 92 \quad \Leftrightarrow \quad n \le 10.$$

The largest allowed prefix length is **10**. It uses exactly $8(10) + 12 = 92$ scalars, while length 11 would require 100. This accounting covers the history carried into the next step. It leaves model weights and temporary calculation buffers outside the budget.

9. Separate delay from interference

Start a history with *The first code for delivery P is 517.* End it with *What was the first code for delivery P?* A model can fail because many positions separate the question from the answer, or because another code looks like a plausible answer. Build two groups of histories to vary those factors separately.

In the **delay** group, insert 0, 8, 32, or 128 unrelated messages between the initial code and the question. Use messages such as *The truck is parked*, with no codes and no changes to delivery P. Use the same initial code and question across the four lengths within a matched set, then repeat with many randomly assigned codes. Keep the number of competing codes fixed at zero. Record the true distance in tokens between the initial code and the question. One extra message can contain several tokens.

In the **interference** group, hold the message count and token length fixed. For example, use 32 equal-length slots. Fill 0, 4, 8, or 16 slots with code-like messages about other deliveries. Fill the remaining slots with unrelated messages of the same token length. Keep the first code and the question at the same positions. Use the model's tokenizer on the full formatted prompt to verify equal length and the same code and question offsets. Distribute competing codes across positions and balance which values appear as answers and distractors. A distractor such as *Code 654 belongs to delivery Q* looks plausible but does not answer the question about P.

Use the same answer format and scoring rule in both groups. Report exact first-code accuracy by delay and by competitor count, along with the number of histories per condition. Inspect the actual input after formatting and truncation to confirm that the initial code reaches the model. Classify wrong answers into copied competitor codes, other wrong codes, and missing answers. A rise in copied competitors when length is fixed is evidence of interference. A drop as irrelevant material is added, with the code still present and no competitors, points to sensitivity to the longer separation. Neither pattern identifies a particular internal operation without further tests.

10. Compose an update and a retrieval

Imagine a dispatcher records an address for each of three deliveries. Later messages choose which delivery is active. The question asks for the address, so the model first needs the active ID and then the address associated with that ID.

Event	Message
1	Store address *12 Oak Street* for delivery A.
2	Store address *95 Pine Street* for delivery B.
3	Store address *7 Elm Street* for delivery C.
4	Make delivery A active.
5	Make delivery C active.
6	Make delivery B active.
7	Prepare the active delivery.

Ask *What is the address of the final active delivery?* The last activation sets the active ID to B. The address lookup for B returns *95 Pine Street* from event 2. The last event contains neither the ID nor the address.

One sufficient summary is a table from delivery IDs to addresses and a field for the current active ID. Address events update the table, and activation events overwrite the field. At query time, read the address under the active ID. A route-only summary that stores B lacks the address. A summary that stores just the latest address gives *7 Elm Street*, which is for C. An archive can also solve the task by finding the last activation and looking up the corresponding earlier address.

Create a matched stream that changes B's address to *40 Maple Street* in event 2 while leaving events 6 and 7 identical. The answer must change to *40 Maple Street*. This tests address retrieval while keeping the final active ID fixed. In another pair, keep the addresses fixed and change event 6 to activate A. The answer then changes to *12 Oak Street* even though the last event stays the same. Randomize addresses and activation order across examples so the final ID is not systematically tied to the most recent address. Keep the address question as the primary task. In a separate diagnostic call with the same history, ask for the active ID. A wrong address alone leaves open whether the tracking step or the lookup step failed.

11. Plan an ablation

Use a synthetic delivery task with two kinds of question. Some histories ask for the first code after a delay. Others ask for the address of the final active delivery after several updates. Generate the training and held-out streams with the same rules for both models, reserving distinct histories and answer combinations for evaluation. Test several lengths below and above the attention window and vary the number of competing codes. Score exact answers separately for retrieval and for tracking followed by retrieval.

Start with a stack of causal window-attention blocks, each with window width W. In the second model, replace the mixer in a fixed subset of layers with a recurrent mixer. Keep W unchanged in the other attention layers. Preserve the tokenizer, embedding width, layer count, feed-forward blocks, and prediction head. Train both models on the same examples for the same number of tokens, using the same optimizer settings and matched random seeds. Use the same validation protocol and report any differences in parameter count or training compute. Replacing a mixer can change both even when the rest of the stack is identical.

Benchmark inference on the same device and runtime with matched batch size and numerical precision. Warm up both models, use the same input lengths and decode lengths, and record the kernels used for each mixer. Measure **prefill** from the start of processing the input prefix on device to availability of the first output scores. Measure **decoding** separately as the time per subsequent generated token. Synchronize the device at the measurement boundaries and report the median and a tail percentile over repeated runs. If needed, use the same prescribed continuation for timing so that model choices about when to stop do not alter the amount of work. Feed that continuation one token at a time using the existing history buffers to measure incremental decoding.

At each prefix length, measure **persistent history** in bytes per sequence. Include the retained KV tensors in the attention layers and any state used to continue the recurrent layers. Check the actual buffers and data types. A windowed layer can retain at most W positions for a streaming cache if it evicts older positions. Also report peak working memory separately, since temporary buffers and weights contribute to peak use. Report quality, time, and bytes together for each length. The result depends on the chosen recurrent mixer, layer placement, and training budget. It measures this replacement in the tested stack and task.

12. Transfer the evaluation idea

Take the shop forecast from Chapter 27. It makes a three-day prediction before the first target day. Suppose the table's weather field contains the latest value for each target date. If a weather forecast is replaced with the measurement after that date, a backtest can read a value unavailable to the original forecaster. Its score can exaggerate what the system could have achieved at the time.

Use a chronological training, validation, and test split. Assign a forecast window to a split only when all of its target days fall inside that split. At each test origin, reconstruct the weather forecasts issued and received before the cutoff. Keep the valid date of each weather prediction distinct from its issue time. For example, a Monday morning forecast for Wednesday may read the weather forecast for Wednesday that arrived by Monday morning. It cannot read an update issued on Tuesday or the measurement made on Wednesday. Fit data transformations on the training period, and freeze model settings after validation.

Add a paired feature-availability check at the same test origins. Run the feature builder on the archived data. Then make a copy that changes only values in weather records unavailable at that origin, leaving their timestamps, all available records, and the sales data the same. Compare the feature rows and, with deterministic prediction, the forecasts. In a correct as-of pipeline they agree. A difference shows that the pipeline read something that was unavailable at that origin. Log the selected weather record IDs and compare them with the expected as-of records to check that the right versions were read.

After the check passes, evaluate the original as-issued forecasts against the observed sales. Report MAE separately for horizons one, two, and three and count origins with missing forecast versions. The paired check tests whether the information boundary holds. The rolling score measures prediction accuracy with that boundary in place.